

23 | Rust与大模型：用 Candle 做一个聊天机器人

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们来聊一聊如何用 Rust 做一个基于大语言模型的聊天机器人。

大语言模型（LLM）是 2023 年最火的领域，没有之一。这波热潮是由 OpenAI 的 ChatGPT 在今年上半年发布后引起的，之后全世界的研究机构 and 科技公司都卷入了大模型的竞争中。目前业界应用大模型训练及推理的主要语言是 Python 和 C/C++。Python 一般用来实现上层框架，而 C/C++ 一般起底层高性能执行的作用，比如著名的框架 PyTorch，它的上层业务层面是用 Python 写的，下层执行层面由 C 执行，因为 GPU 加速的部分只能由 C/C++ 来调用。

看起来大语言模型好像和 Rust 没什么关系，必须承认，由于历史积累的原因，在 AI 这一块儿 Rust 的影响力还非常小。但从另一方面来讲呢，Rust 是目前业界除 Python、C++ 外，唯一有潜力在未来 20 年的 AI 发展中发挥重要作用的语言了。为什么这么说呢？

首先 Rust 的性能与 C/C++ 一致，并且在调用 GPU 能力方面也同样方便；其次，Rust 强大的表达能力，不输于 Python，这让人们使用 Rust 做业务并不难；然后，Rust 的 cargo 编译

成单文件的能力，以及对 WebAssembly 完善的支持，部署应用的时候非常方便，这比 Py + C/C++ 组合需要安装的一堆依赖和数 G 的库方便太多。

目前 Rust 生态中其实已经有很多 AI 相关的基础设施了，你可以从我给出的 [🔗链接](#) 里找到。

世界上最大的机器学习模型仓库平台 HuggingFace（机器学习领域的 Github）推出了 Rust 机器学习框架 [🔗Candle](#)。在这个官方代码仓库上，HuggingFace 上解释了为什么要做一个 Rust 的机器学习框架。

1. Candle 上云端的 Serverless 推理可行。PyTorch 那一套体积太大，安装完得几个 G，而 Candle 编译后的可执行文件才十几 M 到几十 M。
2. Candle 可以让你避免 Python 的 [🔗GIL](#)，从而提高性能。
3. HuggingFace 已经用 Rust 写了不少基础工具了，比如 [🔗safetensors](#) 和 [🔗tokenizers](#)。

Elon Musk 的 [🔗x.ai](#) 发布后，页面上也有一段对 Rust 的溢美之词：Rust 被证明是一个理想的选择，用于构建可扩展的、可靠的、可维护性的基础设施。它提供了高性能、丰富的生态系统，它能阻止大部分错误，这些错误在分布式系统中经常会碰到。由于我们的团队规模很小，基础设施的可靠性就显得至关重要，否则维护工作会浪费大量创新的时间。Rust 给我们提供了信心，任何的代码修改或重构都可以产出可工作的程序，并且在最小监管下可以持续运行好几个月（而不会出问题）。

Musk 甚至说 Rust 语言是未来 [🔗构建 AGI](#) 的语言。

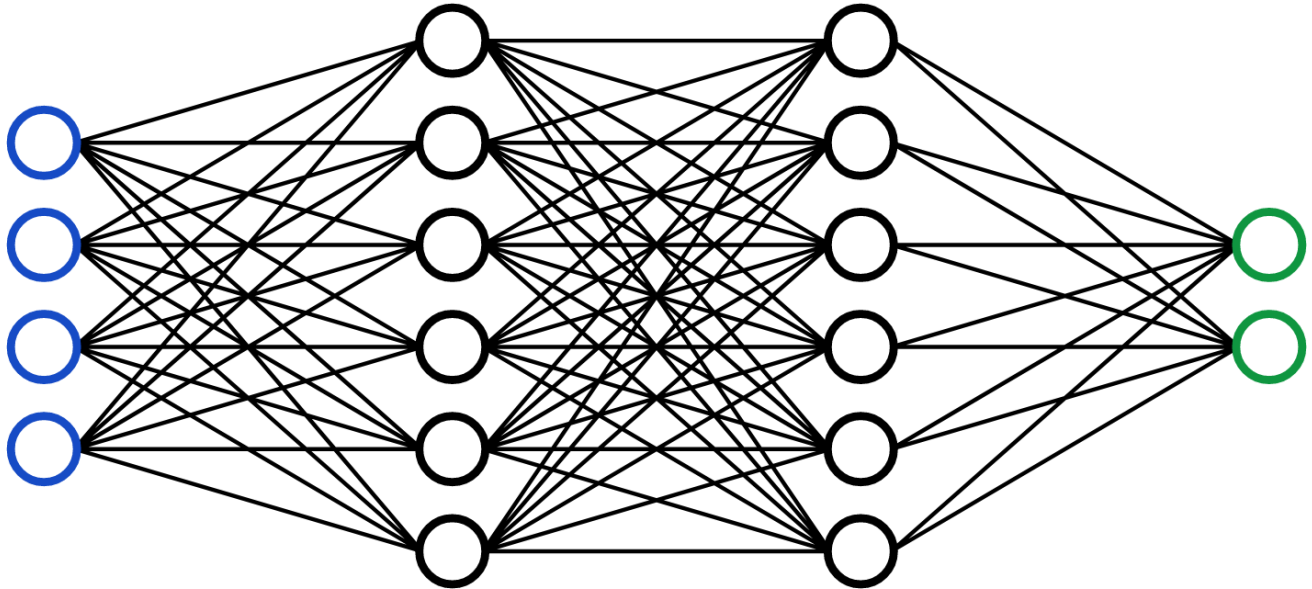
在使用 Rust 尝试做聊天机器人之前，我们先来了解一下相关的背景知识。

大语言模型背景知识

这节课我们还是主要讲 Rust 及 Rust 的应用，所以相关的背景知识我们就简单概括一下。

机器学习（Machine Learning，ML）泛指用计算机对数据集按照一定的算法进行数据处理、分析、聚类、回归等。在执行前，人往往不知道结果是什么，所以叫机器学习。机器学习可以用于自动提取信息，自动或辅助人类做决策。

而神经网络（Neural Network）是机器学习的一类算法，它是模拟人的大脑神经元和连接的一种算法。目前整个业界投入资源最多的就是在这类算法上面，在这个类别中的创新也是最多的，有种观点认为神经网络算法是通向真正的 AI 最可能正确的路径。



图片来源：<https://victorzhou.com/media/nn-series/network.svg>

而深度学习（Deep Learning），其实就是层次很多很深的神经网络。你可以想象，层次越深，节点数（图里那些圈）越多，就越能模拟大脑。但是深度学习带来的问题就是，层次越深，节点数越多，则那些线（就是权重值）就会越多，呈指数级增长，那么计算量就会越来越大。其实之前十几、二十年 AI 进展不大，主要就是因为这个计算能力限制了。

通过无数人的探索，我们发现可以把深度学习的计算并行化，优化了很多工程上的算法。这样就能够在大家熟悉的主要用来玩游戏的 GPU 显卡上运行，大大提高了计算速度。这个视角，二十多年前 Nvidia 的老黄就看明白了，早早地提供好了基础设施，CUDA、CUDNN 等。所以现在你就看到了，一卡难求，已经严重影响到了游戏玩家的生存。

ChatGPT 与 LLaMA

那么 ChatGPT 是什么呢？英文是 Chat Generative Pre-trained Transformer，是 OpenAI 提供的在线 AI 对话服务，具有令人惊叹的理解能力和回答能力。你可以把 ChatGPT 理解成

一个为对话调优的预训练转换模型。GPT 3 有 1750 亿参数，有传言 GPT 4 有 1.76 万亿参数。参数是什么，就是图中节点之间的连线，你可以想象一下 1750 亿根线的场景。

OpenAI 搞出了 ChatGPT，让全世界惊掉了下巴，但是它是闭源的。于是 Meta 公司（前 Facebook）的 Yann LeCun（图灵奖得主）团队，搞了一个开源版本的 GPT，叫 LLaMA。它也是在巨量的源数据集上进行的训练，生成了 70 亿（7B），130 亿（13B），650 亿（65B）三个版本参数的大模型文件，可以供业界做学习和研究使用。现在已推出 LLaMA 2，正在做 LLaMA 3。

LLaMA 搞出来后，掀起了 LLM 界的狂欢。全世界的团队在 LLaMA 的基础上，继续调优，推出了各种各样的大模型。比如国内清华的 ChatGLM，零一万物的 Yi。国外的 Mistral、OpenChat、Starling 等。这块儿非常卷，大家都在争相推出自己调优后的版本，每天早上睡醒起来，都发现又推出了几个新的 LLM。

这些训练好后的文件一般从几个 G 到几十 G 不等，也有几百 G 的。要运行它们，得有非常强大的机器才行。比如 7B 的 LLaMA 2 文件，每个权重为一个 f16 浮点数，占两个字节，所以可以估算出要运行 LLaMA 2 模型，起码得有 14G 的内存或显存。内存还好，显存超过 14G 的个人用户真不多。并且，LLaMA2 的模型文件是 PyTorch 导出的，只能由 PyTorch 框架来运行。

llama.cpp 与量子化方法

车到山前必有路，大神 Georgi Gerganov 搞了一个项目：[🔗 llama.cpp](#)。它是一个用 C/C++ 重新实现引擎的版本，不需要安装 PyTorch，就可以运行 LLaMA 2 模型文件。最关键的是，它提出了一种量子化（quantization）方法，可以将权重从 16 位量子化到 8 位、6 位、5 位、4 位，甚至 2 位。这样，就相当于等比缩小了占用内存的规模。比如，一个 4 位量子化版本的 LLaMA 2 7B 模型，就只需要不到 4G 的内存 / 显存就能运行。这样，就能适配大多数的个人计算机了。

这种量子化方法是个重大创新，它直接促进了 LLM 生态的进一步繁荣。现在 HuggingFace 上有大量量子化后的模型，比如 [🔗 openchat_3.5.Q4_K_M.gguf](#) 就是一个 OpenChat 的 4 位量子化的版本。我们下载的时候，直接下载这些量子化后的模型文件就可以了。

请注意，这些文件是训练后的成品，我们下载它是用来做推理（infer）的，而不是训练（train）的。当然，我们可以在这些成品模型上运行调优（fine tune）。

大模型文件格式

目前 HuggingFace 上有几种常见的 LLM 文件格式。

bin 格式：Pytorch 导出的模型文件格式

safetensors 格式：HuggingFace 定义的一种新的模型文件格式，有可能成为未来的主流格式。HuggingFace 用 Rust 实现 safetensors 格式的解析，并导出为 Py 接口，请参见 [🔗 链接](#)。

ggml 格式：llama.cpp 项目量子化模型的前期模型格式。

gguf 格式：llama.cpp 项目量子化模型的后期模型格式，也是现在主流的量子化 LLM 格式。

Rust 的机器学习框架

Rust 生态现在有几个比较不错的 ML 框架，最好的两个是：🔗 [Candle](#) 和 🔗 [burn](#)。后续，我们以 Candle 为例来介绍。

Candle 介绍

据 Candle 官网介绍，它是一个极小主义机器学习框架，也就是没什么依赖，不像 Pytorch 那样装一堆东西，部署起来很麻烦。但其实它也能用来训练。

它有下面这些特性：

HuggingFace 出品。近水楼台先得月，Candle 几乎能支持 HuggingFace 上所有的模型（有的需要经过转换）。

语法简单，跟 PyTorch 差不多。

CPU、Cuda、Metal 的支持。

让 serverless 和快速部署成为可能。

模型训练。

分布式计算（通过 NCCL）。

开箱即用的模型支持，LLaMA、Whisper、Falcon 等等。

Candle 不仅仅是大模型深度学习框架，它还是一个机器学习框架，因此它也支持其他的机器学习算法和强化学习（reinforcement learning）。下面我们就来看看如何利用 Candle 框架做一个聊天机器人。

注：这节课的代码适用于 candle_core v0.3 版本。

使用 Candle 做一个聊天机器人

下载模型文件

我对一些大模型进行了测试，发现 OpenChat 的对话效果比较好，所以下面我们用 OpenChat LLM 来进行展示。我们会用 [quantized 8bit](#) 的版本。其实 4bit 的版本也是可以的，效果也非常好。

Candle 官方的示例比较复杂，我为这个课程定制了一个更简单的独立运行的 [示例](#)。你可以将这个仓库克隆下来，进入目录。在运行代码之前，下载模型文件和 tokenizer.json 文件。

 复制代码

```
1 与代码目录同级的位置，创建一个目录
2 mkdir hf_hub
3 进入这个目录，请下载
4 https://huggingface.co/TheBloke/openchat_3.5-GGUF/blob/main/openchat_3.5.Q8_0.ggu
5 和
6 https://huggingface.co/openchat/openchat_3.5/blob/main/tokenizer.json
7 将这个 tokenizer.json 重命名为 openchat_3.5_tokenizer.json
```

目录结构：

 复制代码

```
1 23-candle_chat/  
2     Cargo.toml  
3     src/  
4 hf_hub/  
5     openchat_3.5_tokenizer.json  
6     openchat_3.5.Q8_0.gguf
```

运行演示

然后，进入 23-candle_chat/ 运行：

 复制代码

```
1 cargo run --release --bin simple
```

出现如下界面，就可以聊天了。

```
Finished release [optimized] target(s) in 0.04s  
Running `target/release/simple`  
avx: false, neon: false, simd128: false, f16c: false  
loaded 291 tensors (7695139840bytes) in 0.02s  
model built  
> █
```

下面是我问的一个问题，bot 的回答好像有点问题，这个模型用英文问的效果会好一些。

```
Finished release [optimized] target(s) in 0.04s  
Running `target/release/simple`  
avx: false, neon: false, simd128: false, f16c: false  
loaded 291 tensors (7695139840bytes) in 0.02s  
model built  
> 请告诉我中国历史朝代的时间  
bot: 中国历史有许多朝代，以下是其时间顺序：  
  
1. 先秦时期（公元前6200年至公元前221年）：这个时期包括商、周、战国等时代。具体的年份有：  
- 商朝(公元前2550年-公元前1600年)  
- 周朝(公元前1600年-公元前771年)  
- 战国时期(公元前770年-公元前221年)  
  
2. 春秋战国时期(公元前475年-公元前221年)：是先秦时期的一个子时期。  
  
3. 秦朝(公元前221年-公元前206年)：古代中国第一个统一全国的帝国，由李斯领导成立。  
  
4. 汉朝(公元前202年-公元9年)：汉朝是汉人创建的第二个统一全国的强盛帝国。历代武尊，包括孔子、尧舜、刘备等著名人士在内。  
  
5. 西汉(公元前202年-公元8年)：是汉朝的早期。  
  
6. 东汉(公元9年-公元220年)：是汉朝的晚期。  
  
7. 三国时期(公元220年-公元280年)：三国时期是äeight Han█
```

代码讲解

你可以看一下代码。

 复制代码

```
1  #![allow(unused)]
2
3  use std::fs::File;
4  use std::io::Write;
5  use std::path::PathBuf;
6  use tokenizers::Tokenizer;
7
8  use candle_core::quantized::gguf_file;
9  use candle_core::utils;
10 use candle_core::{Device, Tensor};
11 use candle_transformers::generation::LogitsProcessor;
12 use candle_transformers::models::quantized_llama as quantized_model;
13
14 use anyhow::Result;
15
16 mod token_output_stream;
17 use token_output_stream::TokenOutputStream;
18
19 struct Args {
20     tokenizer: String,
21     model: String,
22     sample_len: usize,
23     temperature: f64,
24     seed: u64,
25     repeat_penalty: f32,
26     repeat_last_n: usize,
27     gqa: usize,
28 }
29
30 impl Args {
31     fn tokenizer(&self) -> Result<Tokenizer> {
32         let tokenizer_path = PathBuf::from(&self.tokenizer);
33         Tokenizer::from_file(tokenizer_path).map_err(anyhow::Error::msg)
34     }
35
36     fn model(&self) -> Result<PathBuf> {
37         Ok(std::path::PathBuf::from(&self.model))
38     }
39 }
40
41 fn main() -> anyhow::Result<()> {
42     println!(
```

```
43     "avx: {}, neon: {}, simd128: {}, f16c: {}",
44     utils::with_avx(),
45     utils::with_neon(),
46     utils::with_simd128(),
47     utils::with_f16c()
48 );
49
50 let args = Args {
51     tokenizer: String::from("../hf_hub/openchat_3.5_tokenizer.json"),
52     model: String::from("../hf_hub/openchat_3.5.Q8_0.gguf"),
53     sample_len: 1000,
54     temperature: 0.8,
55     seed: 299792458,
56     repeat_penalty: 1.1,
57     repeat_last_n: 64,
58     gqa: 8,
59 };
60
61 // load model
62 let model_path = args.model()?;
63 let mut file = File::open(&model_path)?;
64 let start = std::time::Instant::now();
65
66 // This is the model instance
67 let model = gguf_file::Content::read(&mut file)?;
68 let mut total_size_in_bytes = 0;
69 for (_, tensor) in model.tensor_infos.iter() {
70     let elem_count = tensor.shape.elem_count();
71     total_size_in_bytes +=
72         elem_count * tensor.ggml_dtype.type_size() / tensor.ggml_dtype.blck_s
73 }
74 println!(
75     "loaded {:?} tensors ({}bytes) in {:.2}s",
76     model.tensor_infos.len(),
77     total_size_in_bytes,
78     start.elapsed().as_secs_f32(),
79 );
80 let mut model = quantized_model::ModelWeights::from_gguf(model, &mut file)?;
81 println!("model built");
82
83 // load tokenizer
84 let tokenizer = args.tokenizer()?;
85 let mut tos = TokenOutputStream::new(tokenizer);
86 // left for future improvement: interactive
87 for prompt_index in 0.. {
88     print!("> ");
89     std::io::stdout().flush()?;
90     let mut prompt = String::new();
91     std::io::stdin().read_line(&mut prompt)?;
```

```

92     if prompt.ends_with('\n') {
93         prompt.pop();
94         if prompt.ends_with('\r') {
95             prompt.pop();
96         }
97     }
98     let prompt_str = format!("User: {prompt} <|end_of_turn|> Assistant: ");
99     print!("bot: ");
100
101     let tokens = tos
102         .tokenizer()
103         .encode(prompt_str, true)
104         .map_err(anyhow::Error::msg)?;
105
106     let prompt_tokens = tokens.get_ids();
107     let mut all_tokens = vec![];
108     let mut logits_processor = LogitsProcessor::new(args.seed, Some(args.temp)
109
110     let start_prompt_processing = std::time::Instant::now();
111     let mut next_token = {
112         let input = Tensor::new(prompt_tokens, &Device::Cpu)?.unsqueeze(0)?;
113         let logits = model.forward(&input, 0)?;
114         let logits = logits.squeeze(0)?;
115         logits_processor.sample(&logits)?
116     };
117     let prompt_dt = start_prompt_processing.elapsed();
118     all_tokens.push(next_token);
119     if let Some(t) = tos.next_token(next_token)? {
120         print!("{t}");
121         std::io::stdout().flush()?;
122     }
123
124     let eos_token = "<|end_of_turn|>";
125     let eos_token = *tos.tokenizer().get_vocab(true).get(eos_token).unwrap();
126     let start_post_prompt = std::time::Instant::now();
127     let to_sample = args.sample_len.saturating_sub(1);
128     let mut sampled = 0;
129     for index in 0..to_sample {
130         let input = Tensor::new(&[next_token], &Device::Cpu)?.unsqueeze(0)?;
131         let logits = model.forward(&input, prompt_tokens.len() + index)?;
132         let logits = logits.squeeze(0)?;
133         let logits = if args.repeat_penalty == 1. {
134             logits
135         } else {
136             let start_at = all_tokens.len().saturating_sub(args.repeat_last_n
137                 candle_transformers::utils::apply_repeat_penalty(
138                     &logits,
139                     args.repeat_penalty,
140                     &all_tokens[start_at..],

```

```

141         )?
142     };
143     next_token = logits_processor.sample(&logits)?;
144     all_tokens.push(next_token);
145     if let Some(t) = tos.next_token(next_token)? {
146         print!("{t}");
147         std::io::stdout().flush()?;
148     }
149     sampled += 1;
150     if next_token == eos_token {
151         break;
152     };
153 }
154 if let Some(rest) = tos.decode_rest().map_err(candle_core::Error::msg)? {
155     print!("{rest}");
156 }
157 std::io::stdout().flush()?;
158 let dt = start_post_prompt.elapsed();
159 println!(
160     "\n\n{:4} prompt tokens processed: {:.2} token/s",
161     prompt_tokens.len(),
162     prompt_tokens.len() as f64 / prompt_dt.as_secs_f64(),
163 );
164 println!(
165     "{sampled:4} tokens generated: {:.2} token/s",
166     sampled as f64 / dt.as_secs_f64(),
167 );
168 }
169
170 Ok(())
171 }

```

我们分段讲解这 100 多行代码。

第 19 行，定义了 Args 参数，这是模型必要的参数配置定义。第 42~48 行，看看有哪些 CPU 特性支持。第 50~59 行，实例化 Args，这些参数都是硬编码进去的，除了两个文件的路径外，需要 LLM 相关知识才能理解。

第 62~81 行，加载大模型文件，并生成模型对象。我们这个模型是 GGUF 格式的，因此需要用 gguf_file 模块来读。Tensor 是 LLM 中的重要概念，它是一个多维数组，可以在 CPU 和 GPU 上计算，在 GPU 上还可以并行计算。一个大模型由很多的 Tensor 组成。我们这个模型中，加载进来了 291 个 Tensor。

第 84 行，加载 tokenizer.json 文件，并生成 tokenizer 实例。tokenizer 用于将输入和输出的文本转化为 Tensor，变成大模型可理解的数据。第 85 行创建 token 输出流实例。第 89 ~ 99 行，建立问答界面，输入提示符为一个 > 号，输出为 bot: 开头。第 101 ~ 104 行将输入的问答转化为 tokens。这个 tokens 就是 Tensor 实例。

第 106 ~ 122 行，是用于处理输入，大模型对输入的 token 做一处理，你可以理解成大模型对你的输入问题先要进行一下理解，然后后面才能做出对应的回答。

LogitsProcessor 是一个用于修改模型输出概率分布的工具。我们可以看到，这个过程中使用的设备写死了，用的 CPU。

第 124 行，"<|end_of_turn|>" 是 OpenChat 模式定义的一轮对话结束的标志。第 125 ~ 156 行，就是对问题的回答。用的设备仍然为 CPU，我们可以猜测应该会很慢。它这里面还有对 penalty 机制的处理。细节也需要去查阅大模型 NLP 相关的知识。

第 158 ~ 167 行，是对本次处理性能的一个汇总。在我的电脑上，纯用 CPU 计算的话，只能达到 1 秒一个多 token 的速度，非常卡。有 GPU 加持的话，会快很多。

你可能发现了，第 87 行有个循环，它是用来实现 interactive 交互效果的，问完一句，回答完，还可以问下一句。

添加命令行参数

前面的 simple 示例，我们所有的参数都是写死在代码里面的。这样方便理解，但不方便使用。我们可以尝试为它添加命令行参数功能。

在 Rust 中，写一个命令行非常简单，直接用 clap，改几行代码就可以了。将上面示例中的 Args 结构体的定义变成下面这样就可以了，然后在调用的时候使用 Args::parse() 生成 Args 实例。

 复制代码

```
1 #[derive(Parser, Debug)]
2 #[command(author, version, about, long_about = None)]
3 struct Args {
```

```

4   #[arg(long, default_value = "../hf_hub/openchat_3.5_tokenizer.json")]
5   tokenizer: String,
6   #[arg(long, default_value = "../hf_hub/openchat_3.5.Q8_0.gguf")]
7   model: String,
8   #[arg(short = 'n', long, default_value_t = 1000)]
9   sample_len: usize,
10  #[arg(long, default_value_t = 0.8)]
11  temperature: f64,
12  #[arg(long, default_value_t = 299792458)]
13  seed: u64,
14  #[arg(long, default_value_t = 1.1)]
15  repeat_penalty: f32,
16  #[arg(long, default_value_t = 64)]
17  repeat_last_n: usize,
18  #[arg(long, default_value_t = 8)]
19  gqa: usize,
20 }
21
22 fn main() {
23     // ...
24     let args = Args::parse();
25     // ...
26 }

```

经过升级的命令有了下面这些参数:

 复制代码

```

1 $ cargo run --release --bin cli -- --help
2   Finished release [optimized] target(s) in 0.04s
3   Running `target/release/cli --help`
4 avx: false, neon: false, simd128: false, f16c: false
5 Usage: cli [OPTIONS]
6 Options:
7     --tokenizer <TOKENIZER>          [default: ../hf_hub/openchat_3.5_tokeniz
8     --model <MODEL>                  [default: ../hf_hub/openchat_3.5.Q8_0.gg
9  -n, --sample-len <SAMPLE_LEN>      [default: 1000]
10    --temperature <TEMPERATURE>      [default: 0.8]
11    --seed <SEED>                    [default: 299792458]
12    --repeat-penalty <REPEAT_PENALTY> [default: 1.1]
13    --repeat-last-n <REPEAT_LAST_N>   [default: 64]
14    --gqa <GQA>                      [default: 8]
15  -h, --help                          Print help
16  -V, --version                       Print version

```

是不是很方便？谁说 Rust 生产力不行的！

点击 [🔗 这里](#) 可以找到源文件，你可以本地试试跑起一个大模型对话机器人。另外，Candle 官方仓库中的 [🔗 示例](#) 功能更强大，但也更复杂，你可以继续深入研究。

小结

这节课我们一起探索了使用 Rust 利用 Candle 机器学习框架开发一个大模型聊天机器人的应用。Rust 目前在 AI 界虽然还不够有影响力，但是未来是相当有潜力的，这也是为什么 HuggingFace 带头出一个 Rust 机器学习框架的原因。

不过这节课我们只是讲了怎么用起来，而如果要深入下去的话，机器学习、深度学习的基础知识就必不可少。如果你有时间精力的话，你可以深入下去好好补充一下这方面的学术知识，毕竟未来几十年，AI 是一个主要问题，也会是一个主要机会。目前 AI 发展速度太快了，有种学习跟不上业界发展速度的感觉。但是不管怎样，学好基础，永远不会过时。

另外，Rust AI 这块儿，虽然已经小有起色，但是作为生态来讲，空白处还很多。所以这也正是学好 Rust 的机会，Rust 可以在 AI 基建这块做大量的工作，这些工作可以服务于 Rust 社区，也可以服务于 Python 乃至整个 AI 社区。

思考题

你可以在我的示例上继续捣鼓，添加 GPU 的支持，在 Linux、Windows、macOS 多种平台上测试一下。欢迎你在评论区贴出你自己的代码，也欢迎你把这节课分享给其他朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (6)

最新 精选



听雨

2023-12-23 来自四川

老师，我下载到了window下的子Linux中运行，报OS error。请问是不是不支持在这里面运行啊

作者回复：我在wsl中测过，可以的。如果想要wsl支持图形界面应用，需要update到最新版wsl。



1



superggn

2024-01-02 来自北京

mac 2021 款 intel 芯片，系统版本 Monterey 12.5.1 会卡住，model built 之后输入一个 hello 就不动了，要过 5 分钟以上才会有回复

作者回复：对电脑要求有点高



Geek_54dac1

2023-12-15 来自四川

该模型在 Candle 中还暂时不支持在 GPU 上运行，因为 Quantized models on Cuda 还不支持，参考：<https://github.com/huggingface/candle/issues/1250>；避免大家挖坑

作者回复：很棒的探索和分享 👍



无限可能

2023-12-15 来自北京

老师好，大模型小白有几个问题。通过 huggingface-cli scan-cache，扫描了一下 huggingface 下载过的文件，都是 7G 或者更大：

1. 这些模型文件里主要是什么内容，是包括了数据么？为啥会这么大？
2. 类似这么大的文件，一定要下载到本地么，是否可以云部署之类，可以用完销毁。
3. 我理解，我想要运行本节课的demo，这 7G 的空间就是要长期规划出去了吧

作者回复：1. 主要是预训练神经网络模型权重数据

2. 要下载到本地，要加载到显存或者内存中运行

3. 是的

共 3 条评论 >



学水

2023-12-15 来自加拿大

这张感觉主要是给对rust有兴趣的mle，中间一些代码如果不是因为用过pytorch，完全不知道在干啥

作者回复: 是的，如果讲ml的话，又是专门一门课程了。可以在后续继续深入下去。你有pytorch基础就非常有利。👍



刘丹

2023-12-15 来自广东

老师，你好，请问能否推荐几个和 ChatGPT 交互的 Rust SDK？最好也能支持其它 LLM。

作者回复: 有几个还不错的。

<https://github.com/sobelio/llm-chain>

<https://github.com/rustformers/llm>

<https://github.com/jondot/awesome-rust-llm>

当然最强的，还是candle啦。

<https://github.com/huggingface/candle>

